Layer Looping

Per-token layer routing looks powerful in hindsight,
but only a gate trained with the model can use it

Tyler Gee
Kāula · kaula.ai

November 2026

Abstract

Can each token of a pretrained language model choose its own path through the layers, looping some and skipping others, to spend compute where it is needed? We measure this exactly. For every token of a frozen Gemma 4 E2B and of Ouro-1.4B, a looped model, we compute the change in that token's next-token loss under every single-layer edit (repeat, skip, swap, block loop, early exit). In hindsight the best edit per token cuts loss by 0.60–0.80 nats on held-out text at no extra compute, and multi-edit paths raise this to 1.30. None of it is reachable. Ridge and MLP routers, reading the current or the previous token's internals, interpretable or raw features, and training-free entropy and norm rules from prior work all fail to beat "no edit"; the hindsight gain grows with the number of options and sits on tokens the model already gets wrong, the signature of selection rather than signal. Light fine-tuning does not teach a standard model to use a loop. Per-token compute does pay off when the router is trained with the model: Ouro's built-in exit gate beats a fixed compute budget on ARC-Easy by +3.6 points at equal compute (paired $p < 10^{-4}$). A finer per-layer skip/once/twice router trained on frozen Ouro loses to that gate on 4 of 5 benchmarks, and a structured router trained on the exact expected loss recovers the gate to within 0.001 nats but never uses an offered extra half-round. A router can only spend computation the model already knows how to use.

1 Introduction

Adaptive-depth methods—adaptive computation time [10], universal transformers [6], early exiting [26], mixture-of-depths [24], layer dropping [7, 19]—assume tokens differ in how much computation they need. Frozen models tolerate layers being skipped, repeated or reordered [15, 27], and recent training-free methods re-apply layers per token at high-entropy positions [2] or where the hidden-state norm peaks [31]. This suggests a tempting program: treat a pretrained model's layers as a library and let a cheap router pick each token's path.

We test that program directly, with an exact per-token oracle, and find a sharp split. Fitted to a frozen model, routing recovers nothing, although the oracle promises a great deal. Trained with the model, a coarse per-token exit gate gives benchmark gains at matched compute. Our contributions:

1. **An exact per-token oracle** over repeat, skip, swap, block-loop and exit edits in a frozen model, computed for all positions in about one forward pass per edit (Section 2).
2. **A negative result for frozen-model routing** across feature sets, router classes, lag, and training-free baselines, with three checks that the hindsight gain is selection, not signal (Section 3).
3. **A positive result for trained-in routing**: paired benchmark comparisons of Ouro-1.4B's exit gate against fixed compute (Section 4).

4. **A diagnosis of why a finer router loses** to that gate, and a structured router that recovers it but declines extra recurrence (Section 5).

2 Method

Edits and cost. A model with L layers runs them in order $0, \ldots, L-1$. An edit changes that schedule for one token: repeat@$\ell \times k$ ($k \in \{2, 3\}$, cost $k-1$), skip@ℓ (cost -1), swap@ℓ (run $\ell+1$ before ℓ, cost 0), or a static block loop. Cost counts extra layer applications. Gemma 4 E2B ($L = 35$) has 140 such programs, including a mid-block loop (layers 13–16 twice) scaled from the best Gemma-2-2B block of Lys et al. [18]. Ouro-1.4B [32] runs 24 layers for 4 rounds (96 unrolled layers) and was pretrained with a per-token exit gate; on it we add early exits and a fifth round, 387 programs in all.

The diagonal swap. We want token t's next-token loss when only t runs program c and every other token runs normally. Because the model is causal, this can be computed for all positions at once: at each step a query attends to the *base-pass* keys and values of every other position and to its *own* modified key and value on the diagonal. This is exact for each token's own loss. On E2B, layers 15–34 reuse the keys and values of layer 13 or 14; the edited token uses its latest own copy, so edits there reach later layers. CPU tests on tiny random Gemma-2 and Gemma 4 models check every oracle delta against an independent recompute that truncates the sequence at t, for every edit type and for arbitrary multi-edit paths.

Oracle and routers. $\Delta\mathrm{NLL}_t(c) = \mathrm{NLL}_t(c) - \mathrm{NLL}_t(\text{base})$; negative means the edit helps. The oracle takes $\min_c \Delta\mathrm{NLL}_t(c)$, or 0 if nothing helps. A router predicts $\widehat{\Delta\mathrm{NLL}}_t$ and routes to $\arg\min_c [\widehat{\Delta\mathrm{NLL}}_t(c) + \lambda\,\mathrm{cost}(c)]$, or to the base program if the minimum is non-negative; we report the *realized* ΔNLL. The **same-token** router reads token t's base-pass features; the **lagged** router reads token $t-1$'s, which exist before t is processed. Interpretable features are per-layer logit-lens [21] entropy and agreement with the final prediction, residual-update norms and attention entropy; raw features are hidden states. Training-free baselines are an EGLR-like entropy gate [2] (same token or one token early) and a norm-peak repeat [31].

Data and evaluation. Oracles use 20,440 positions of WikiText-103 [20] prose and GSM8K [5] text in alternating 512-token chunks; chunks with index $\equiv 0 \pmod 5$ are held out, and 95% confidence intervals bootstrap over held-out chunks. Ouro router training and loss curves use FineWeb-Edu [22]. Benchmarks are ARC-Easy and ARC-Challenge [4], HellaSwag [30], MMLU [12] and WinoGrande [25]: 1,000 items each, scored by length-normalized log-likelihood. Each answer token follows its own path against the other tokens' base-pass keys and values, the same accounting as Ouro's early-exit evaluation. Comparisons are paired on identical items, with a bootstrap CI over items and McNemar's exact test. Compute is layers per answer token.

3 Frozen-model routing finds nothing

Figure 1 and Table 1 show the gap. On both models the hindsight oracle removes 34–42% of base loss, and every learned router is *worse* than doing nothing, with confidence intervals excluding zero. AUCs at or below chance mean the routers cannot even rank which tokens an edit would help. Swept over λ, every router's best point at zero extra compute is "no edit". Routing from the previous token does no

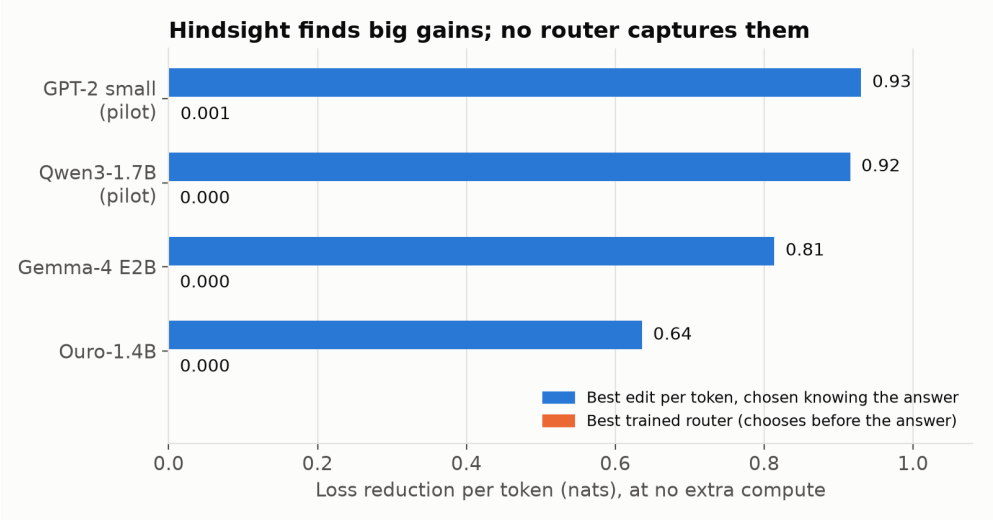


Figure 1: Loss reduction per token at zero extra compute: the best single edit chosen in hindsight (blue) versus the best trained router (orange, zero on every model). Gemma 4 E2B and Ouro-1.4B use the exact per-token oracle over all 20,440 positions; the GPT-2 small and Qwen3-1.7B pilots [23, 29] apply each edit to the whole sequence.

Table 1: Held-out ΔNLL (nats/token, lower is better) at $\lambda = 0$. AUC: detecting tokens an edit helps from the router's predicted best gain (0.5 is chance). Training-free rules never beat 0.

	Gemma 4 E2B (base 1.91)		Ouro-1.4B (base 1.79)	
Policy	ΔNLL [95% CI]	AUC	ΔNLL [95% CI]	AUC
Hindsight oracle	−0.797	—	−0.604	—
Best static program	+0.014	—	+0.002	—
Same-token, interpretable	+0.077 [+0.052, +0.101]	0.20	+0.073 [+0.053, +0.096]	0.40
Lagged, interpretable	+0.068 [+0.056, +0.082]	0.35	+0.057 [+0.046, +0.070]	0.45
Same-token, raw	+0.330 [+0.282, +0.386]	0.45	+0.139 [+0.090, +0.193]	0.47
Lagged, raw	+0.428 [+0.368, +0.493]	0.52	+0.228 [+0.170, +0.273]	0.52
Entropy / norm-peak rules	0.000	—	−0.0001	—

better or worse than from the current one, because there is nothing to recover. Label-free self-selection (run several edits, keep the most confident output) gives at best +0.0008 [−0.0007, +0.0025] on E2B.

The hindsight gain is selection, not signal. Three checks on E2B. *It is not numerical noise*: recomputing the oracle in float32 gives the same −0.81 nats as bf16 (per-edit correlation 0.9999), and routers retrained on float32 labels are unchanged. *It grows with the number of options*: over multi-edit paths, the oracle at zero extra compute rises from −0.87 (single edits) to −0.97 (a 396-path shared library) to −1.30 (per-token beam search), while an MLP controller's best point stays at no edit. *It sits on tokens the model already gets wrong*: none of the most confident fifth of tokens are helped, against all of the least confident three fifths. A per-token oracle over many edits measures how often *some* perturbation happens to favor the right token.

Table 2: Ouro-1.4B accuracy (acc_norm, 1,000 items). Gate: Ouro's own exit gate at threshold 0.2. Δ: gate minus fixed 3 rounds, paired 95% CI and McNemar p. Layers per answer token in parentheses.

Task	4 rounds (96)	3 rounds (72)	Gate	Δ [95% CI]	p
ARC-Easy	0.784	0.746	0.782 (72.6)	+3.6 [+2.2, +5.0]	$< 10^{-4}$
ARC-Challenge	0.516	0.502	0.512 (71.9)	+1.0 [−0.2, +2.2]	0.14
HellaSwag	0.714	0.709	0.711 (65.8)	+0.2 [−0.8, +1.2]	0.84
MMLU	0.669	0.643	0.667 (79.8)	+2.4 [+0.9, +3.9]	0.003
WinoGrande	0.672	0.676	0.671 (72.0)	−0.5 [−2.0, +1.0]	0.60

Light fine-tuning does not teach a loop. On E2B we trained LoRA [13] adapters (rank 16, about 6M tokens) with layers 5–8 looped twice and the loop's output mixed in by a learnable weight α initialized at 0, or by a learned per-token gate. Training kept $|\alpha| < 5 \times 10^{-4}$ even at a 50× learning rate; the gate closed to 0.2%. Every looped variant landed within seed noise of a LoRA control without a loop (WikiText NLL 2.135–2.137 vs 2.136).

4 A router trained with the model works

Ouro was pretrained on 7.7T tokens with a per-token exit gate. Table 2 compares that gate with running a fixed 3 rounds for every token. On ARC-Easy the gate gains 3.6 points at the same compute (72.6 vs 72 layers). On HellaSwag it matches fixed rounds with 6 fewer layers. On MMLU it gains 2.4 points but uses 8 more layers, so that comparison is not compute-matched. On all five tasks it stays within 0.1–0.4 points of full 96-layer accuracy, and a fifth round for every token adds nothing (within 0.8 points of 4 rounds everywhere). On held-out text, the gate at threshold 0.5 exits 54% of tokens early and saves 13.3 layers per token at $\Delta\text{NLL} = -0.002$; frozen-model routers on the same model are worse than no edit (Table 1).

5 Finer control loses to coarse control

A per-layer router. We trained a head at each of Ouro's 96 unrolled layers to choose skip, once or twice per token, with Ouro frozen, straight-through Gumbel-softmax [14] and a budget of 72 layers per token, on about 10M FineWeb-Edu tokens. A Lagrangian budget collapsed to 10–18 layers within 25 steps (one shared signal flips all heads to skip together); a two-sided quadratic penalty with an annealed target was stable. Against Ouro's gate on the benchmarks of Table 2, the router is worse on 4 of 5 tasks while using more layers (79–84): ARC-Challenge −3.2, ARC-Easy −1.8, HellaSwag −3.1, MMLU −4.4 points (all $p < 0.05$); WinoGrande +0.8 (n.s.). Trained jointly with LoRA it matched a LoRA-only control's full-compute MMLU and WinoGrande at 23% less compute, but the fine-tune itself cost the base model 4 MMLU points.

Why it lost. Figure 2 places the router above even fixed compute: 2.433 nats at 78 layers, against 2.368 for 3 fixed rounds at 72 and 2.351 for the gate at 70. Its per-token layer count correlates *negatively* with token difficulty under the base model (Spearman −0.23; the gate: +0.42). The per-token decisions are not noise: replacing them with each layer's majority action raises loss to 2.542, and shuffling them across tokens to 2.573. The cheapest saving is dropping round 4, worth only 0.025 nats per token, and the gate takes it; the router skips none of round 4 and thins round 3 instead, and never chooses "twice".

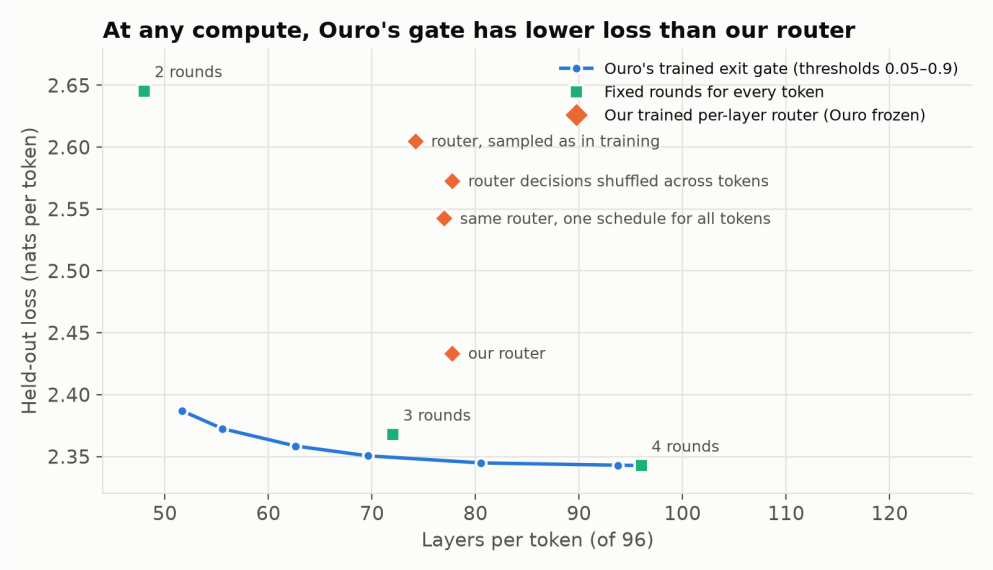


Figure 2: Held-out FineWeb-Edu loss against layers per token on Ouro-1.4B (32 chunks of 1,024 tokens). Ouro's gate (thresholds 0.05–0.9) lies below fixed rounds; our per-layer router lies above both.

Our reading, not yet tested directly: a head deciding one layer at a time sees that skipping any single late layer hurts, and cannot see that skipping all 24 together is nearly free.

A structured router recovers the gate. We rebuilt the router around that diagnosis: at each round boundary a token stops, or stops after re-running the second half of the round once more (+12 layers). With Ouro frozen, every option's per-token loss is fixed, so we train on the exact expected loss, with no sampling, warm-started from Ouro's gate at three compute prices, against a stop-only control. On held-out FineWeb-Edu all variants land on the gate's curve to within 0.001 nats (2.3372 vs 2.3362 at 70 layers per token), about 0.09 nats better than the per-layer router; the training method was the problem. But neither beats the gate, and training drives the extra half-round from 0.25% of tokens to 0.01%; at evaluation it is never used. Re-running half a round raises a token's loss on average by 0.99, 0.93, 0.73 and 0.68 nats after rounds 1–4. It helps 16–20% of tokens by about 0.13 nats each, but those tokens are identifiable only in hindsight.

6 Related work

Per-token computation in frozen models. EGLR [2] re-applies the top layers at high-entropy tokens and the Tell-Tale Norm [31] recurses layers guided by hidden-state norms. Think-at-Hard [8] adds whole-model iterations for hard tokens with a trained backbone. **Programs of layers.** CoLa [16] searches per-sample skip/repeat chains, POLAR [17] learns programs of layers, and Dr.LLM [11] trains sequence-level routers. **Static loops.** Inner Loop Inference [18] and Training-Free Looped Transformers [3] loop fixed blocks in frozen models, with gains within noise. **Routers versus oracles.** Glavas et al. [9] find per-token skip controllers no better than a constant input, Duo-LLM [1] finds large gaps between oracles and learned routers, and Wei et al. [28] argue skip headroom is shrinking in modern models. Our results extend these to loops, swaps and multi-edit paths, with an exact oracle, and contrast them with a gate trained in. **Looped models.** Ouro [32] pretrains recurrence with a learned exit; universal

transformers [6] and ACT [10] are its antecedents.

7 Limitations

Main results use 1.4B–2B-parameter models, and the per-token oracles are 20k-token runs. Our routers trained on about 10M tokens, against 7.7T for Ouro's gate. Benchmark scoring has each answer token attend to the other tokens' base-pass keys and values; this is exact for its own prediction and matches Ouro's evaluation, but is not a deployed decoding loop. Compute is counted in layer applications, not wall-clock time.

8 Conclusion

Per-token compute allocation is real, but it has to be learned jointly with the model. Fitted to a frozen network, routers find nothing, although a hindsight oracle promises large gains; a gate trained with the network finds benchmark gains at matched compute. The useful granularity so far is coarse: "how many rounds" beat "which layer, how many times", although the second contains the first. A router can choose how much of a model's trained recurrence each token gets, but new kinds of looping have to be trained into the model itself.

Code and data. https://github.com/Kaula-AI/layer-looping

References

[1] Keivan Alizadeh, Iman Mirzadeh, Hooman Shahrokhi, Dmitry Belenko, Frank Sun, Minsik Cho, et al. Duo-LLM: A framework for studying adaptive computation in large language models. *arXiv preprint arXiv:2410.10846*, 2024.

[2] Soham Bhattacharjee, Dushyant Singh Chauhan, Salem Lahlou, Martin Takac, and Nils Lukas. Entropy-gated latent recursion. *arXiv preprint arXiv:2606.16620*, 2026.

[3] Lizhang Chen, Jonathan Li, Chen Liang, Ni Lao, and Qiang Liu. Training-free looped transformers. *arXiv preprint arXiv:2605.23872*, 2026.

[4] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[6] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

[7] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations*, 2020.

[8] Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, Yu Wang, et al. Think-at-hard: Dynamic looped transformers for improved reasoning. *arXiv preprint arXiv:2511.08577*, 2025.

[9] Theodore Glavas, Joud Chataoui, Florence Regol, Wassim Jabbour, Antonios Valkanas, Boris N. Oreshkin, et al. Dynamic layer selection in decoder-only transformers. *arXiv preprint arXiv:2410.20022*, 2024.

[10] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[11] Ahmed Heakl, Martin Gubri, Salman Khan, Sangdoo Yun, and Seong Joon Oh. Dr.LLM: Dynamic layer routing in LLMs. *arXiv preprint arXiv:2510.12773*, 2025.

[12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[14] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-softmax. In *International Conference on Learning Representations*, 2017.

[15] Vedang Lad, Jin Hwa Lee, Wes Gurnee, and Max Tegmark. The remarkable robustness of LLMs: Stages of inference? *arXiv preprint arXiv:2406.19384*, 2024.

[16] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? test-time depth adaptation of pretrained LLMs. *arXiv preprint arXiv:2507.07996*, 2025.

[17] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? learning program-of-layers in LLMs. *arXiv preprint arXiv:2606.06574*, 2026.

[18] Jonathan Lys, Vincent Gripon, Bastien Pasdeloup, Axel Marmoret, Lukas Mauch, Fabien Cardinaux, et al. Inner loop inference for pretrained transformers: Unlocking latent capabilities without training. *arXiv preprint arXiv:2602.14759*, 2026.

[19] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*, 2024.

[20] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[21] nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

[22] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[24] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

[25] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[26] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.

[27] Qi Sun, Marc Pickett, Aakash Kumar Nain, and Llion Jones. Transformer layers as painters. *arXiv preprint arXiv:2407.09298*, 2024.

[28] Rui Wei, Rui Du, Hanfei Yu, Devesh Tiwari, Jian Li, Zhaozhuo Xu, et al. The diminishing returns of early-exit decoding in modern LLMs. *arXiv preprint arXiv:2603.23701*, 2026.

[29] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[30] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[31] Jinyang Zhang, Hongxin Ding, Yue Fang, Weibin Liao, Muyang Ye, Junfeng Zhao, et al. The tell-tale norm: ℓ_2 magnitude as a signal for reasoning dynamics in large language models. *arXiv preprint arXiv:2606.06188*, 2026.

[32] Rui-Jie Zhu et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.